COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	18/09/2026	**DATE SUBMITTED:**	09/23/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

C++ Code representation of the Tree

```
GeneralTree<char> tree('A');
GeneralTree<char>* b = tree.addChild('B');
GeneralTree<char>* c = tree.addChild('C');
GeneralTree<char>* d = tree.addChild('D');
d->addChild('H');
GeneralTree<char>* e = tree.addChild('E');
e->addChild('I');
GeneralTree<char>* j = e->addChild('J');
j->addChild('P');
j->addChild('Q');
GeneralTree<char>* f = tree.addChild('F');
f->addChild('K');
f->addChild('L');
f->addChild('M');
GeneralTree<char>* g = tree.addChild('G');
g->addChild('N');
```

NODE	HEIGHT	DEPTH
A	3	0
B	2	1
C	2	1
D	2	1
E	2	1
F	2	1

G	2	1
H	1	2
I	1	2
J	1	2
K	1	2
L	1	2
M	1	2
N	1	2
P	0	3
Q	0	3

Pre-order	A B C D H E I J P Q F K L M G N
Post-order	B A C H D I E P J Q K F L M N G
In-order	B C H D I P Q J E K L M F N G A

Task 3: Table 3.1

[TASK 3: 3.2 FUNCTION FOR PRE-ORDER, POST-ORDER AND IN-ORDER.]

Pre-order

```
template <typename T>
void GeneralTree<T>::preOrder() {
    std::cout << key << " ";
    if (firstChild != nullptr) firstChild->preOrder();
    if (nextSibling != nullptr) nextSibling->preOrder();
}
```

Output

```
Pre-order:  A B C D H E I J P Q F K L M G N
```

Task 3: Table 3.2 Pre-order Function

Post-order

```cpp
template <typename T>
void GeneralTree<T>::inOrder() {
    //Traverse the first child (if it exists)
    if (firstChild != nullptr) {
        firstChild->inOrder();
    }

    //Visit the current root node
    std::cout << key << " ";

    //Traversd all remaining siblings sequentially
    if (firstChild != nullptr) {
        GeneralTree<T>* sibling = firstChild->nextSibling;
        while (sibling != nullptr) {
            sibling->inOrder();
            sibling = sibling->nextSibling;
        }
    }
}
```

Output

```
Post-order: B C H D I P Q J E K L M F N G A
```

Task 3: Table 3.2 Post-order Function

In-order

```cpp
template <typename T>
void GeneralTree<T>::postOrder() {
    //Visssit the leftmost child's subtree completely
    if (firstChild != nullptr) {
        firstChild->postOrder();
    }

    //Visit the current node
    std::cout << key << " ";

    //Visit the remaining siblings' subtrees completely
    if (nextSibling != nullptr) {
        nextSibling->postOrder();
    }
}
```

Output

```
In-order:   B A C H D I E P J Q K F L M N G
```

Task 3: Table 3.2 In-order Function

Function for FindData

```cpp
template <typename T>
void GeneralTree<T>::findData(std::string choice, T key) {
    bool found = false;

    if (choice == "PRE") {
        found = preOrderSearch(key);
    } else if (choice == "IN") {
        found = inOrderSearch(key);
    } else if (choice == "POST") {
        found = postOrderSearch(key);
    } else {
        std::cout << "Invalid choice selection!" << std::endl;
        return;
    }

    if (found) {
        std::cout << "{" << key << "} was found!" << std::endl;
    }
}
```

```cpp
template <typename T>
bool GeneralTree<T>::preOrderSearch(T value) {
    if (key == value) return true;
    if (firstChild != nullptr && firstChild->preOrderSearch(value)) return true;
    if (nextSibling != nullptr && nextSibling->preOrderSearch(value)) return true;
    return false;
}

template <typename T>
bool GeneralTree<T>::inOrderSearch(T value) {
    if (firstChild != nullptr && firstChild->inOrderSearch(value)) {
        return true;
    }
    if (key == value) {
        return true;
    }
    if (firstChild != nullptr) {
        GeneralTree<T>* sibling = firstChild->nextSibling;
        while (sibling != nullptr) {
            if (sibling->inOrderSearch(value)) return true;
            sibling = sibling->nextSibling;
        }
    }
    return false;
}
```

```cpp
template <typename T>
bool GeneralTree<T>::postOrderSearch(T value) {
    if (firstChild != nullptr && firstChild->postOrderSearch(value)) {
        return true;
    }
    if (key == value) {
        return true;
    }
    if (nextSibling != nullptr && nextSibling->nextSibling != nullptr) {
        if (nextSibling->postOrderSearch(value)) {
            return true;
        }
    }
    return false;
}
```

Output

```cpp
std::cout << "Searching for 'P' using PRE:  " << std::endl;
tree.findData("PRE", 'P');

std::cout << std::endl;

std::cout << "Searching for 'Z' using POST: " << std::endl;
tree.findData("POST", 'Z'); // No output expected if not found

std::cout << std::endl;

std::cout << "Searching for 'M' using IN:   " << std::endl;
tree.findData("IN", 'M');
```

Task 3: Table 3.3 FindData

Function for Insertion

```cpp
// finding the node or the known subroot to insert the child below it
template <typename T>
GeneralTree<T>* GeneralTree<T>::findNode(GeneralTree<T>* root, T targetKey){
    if (root == nullptr) return nullptr;
    if(root->key == targetKey) return root;

    GeneralTree<T>* found = findNode(root->firstChild, targetKey);
    if (found != nullptr) return found;

    // checks the sibling branches
    return findNode(root->nextSibling, targetKey);

}

template <typename T>
bool GeneralTree<T>::insert(T parentKey, T childValue) {
    GeneralTree<T>* parentNode = findNode(this, parentKey);

    if (parentNode != nullptr) {
        parentNode->addChild(childValue);
        return true;
    }
```

```cpp
std::cout << "---|Insertion|---" << std::endl;
tree.insert('G', 'O');
std::cout << "Pre-order:  "; tree.preOrder();  std::cout << std::endl;
std::cout << "In-order:   "; tree.inOrder();   std::cout << std::endl;
std::cout << "Post-order: "; tree.postOrder(); std::cout << std::endl;
std::cout << "----------------------\n" << std::endl;
return 0;
```

Output

```
---|Insertion|---
Pre-order:  A B C D H E I J P Q F K L M G N O
In-order:   B A C H D I E P J Q K F L M N G O
Post-order: B C H D I P Q J E K L M F N O G A
----------------------
```

Task 3: Table 3.4 Insertion function

Procedural Overall Analysis

In the creation for **Task 1**, which is the creation of a general tree figure, I created an approach to make the certain nodes as a subroot of the root that contains a child. So that in the other functions, such as traversing and inserting function, it will be easier to access the **targetKey** of the subroot to insert a specific **childValue**. The linking of roots flows is as represented:

Tree('A') -> The main root
GeneralTree<char>* b = tree.addChild('B') -> Treated as a (subroot) child of a root
d->addChild('H') -> Treated as a child of the subroot only (function Adds a child to the subroot)

Next is the **Task 2**, which is just a completion of each Node on what height and Depth they are in terms of the general tree provided.

Next, **ILOB, TASK 3**. Which is the code implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3 Table 3.1, Is just the basic flow procedure of each traversal strategy by hand. Which is represented in the table above.

Task 3 Table 3.2, Which is the entire main functionality of the code implementation, the functions for the pre-order, post-order, and in-order traversal to make sure that each function displays an output into the console exactly similar to the Table 3.1. The **pre-order function** has the condition rule that it first visits the **Root**, then recursively visit **each child subtree** from left to right. For example:

A (root) is visited. (print A)
Move to leftmost **child B,** which has no child -> (print B)
Move to **sibling C,** which also has no child -> (print C)
Move to **sibling D** print (D), has a child, therefore move to its **child H**, which has not child -> (print H)
Move to **sibling E**…

Final Print Sequence: A B C D H E I J P Q F K L M G N

In the In-order Traversal, has the rule of visiting the **first child subtree** completely, then visiting the **Root**, then recursively visiting **all remaining sibling subtrees** from left to right. For example:

Start root A its **first child is B.**
Evaluate B -> no children, therefore **print B itself**.
The first child B is done, visit current root A
visit Root (A), print root.

Traverse in the siblings of B sequentially,

C has no children, **print C**.
D has a first child, therefore **visits H**. **Evaluate H**, which has no children, therefore; **print H**.

Visit the root of H, which is D, and **print D**.

Continue the sequence and the Final Print Sequence will be: B A C H D I E P J Q K F L M N G

Last is the Post-order Traversal, which has the rule of **visiting all child subtrees** completely from left to right, then visiting the **root node** last. As the representation:

Start at A, but before we print A, we must process all of its subtrees.

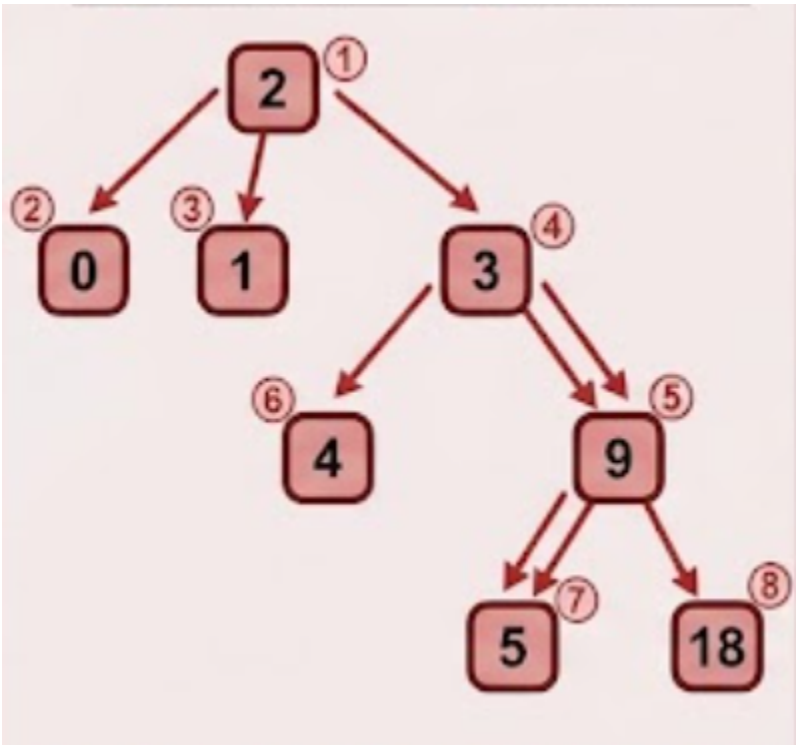
Subtree B has no children, **Print B**.
Subtree C has no children, **Print C**.
Subtree D, has children,. **Therefore it's treated as a sub(root),** must process children first. Which is H.
Subtree H has no children therefore, **print H**.
Now that **children of D are done**, **Print D**.
Subtree E Must process Children first,
'I' has no child, (print I)
J has a child, must **process J's children first**.
P has no child, **Print P,**
Q has no child, **print Q**.
J's children is done, therefore print J.
E children are done, therefore print E…

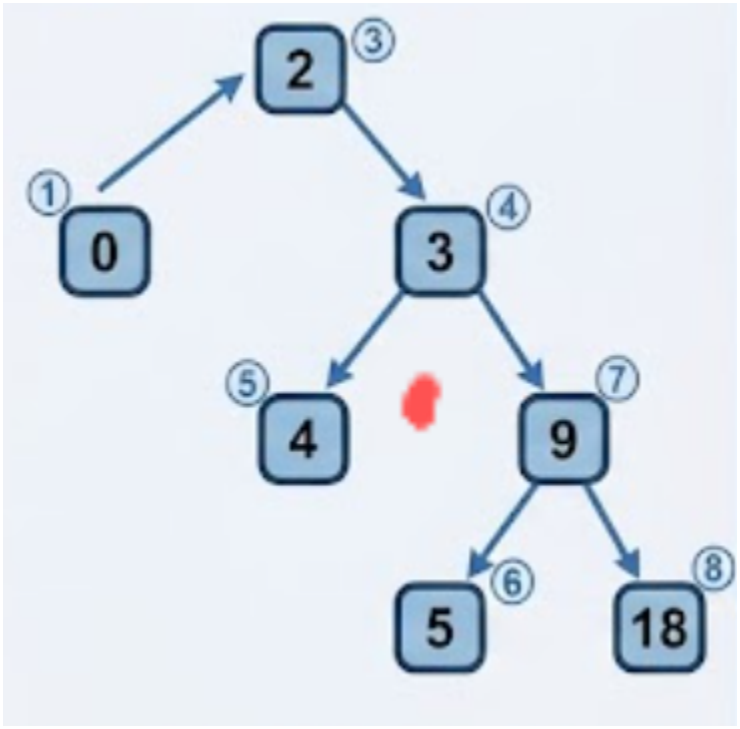
And that's how the function works. In the pre, post, and in order traversal of each function.

In the **Task 3 Table 3.3**, Is the function implementation to find Data in the general tree. This function is to determine the value in its traversal method. It consists of a **helper function** to determine certain conditions such as if the key exists or the certain child is equivalent to nullptr, it will return its conditions to be processed on searching.

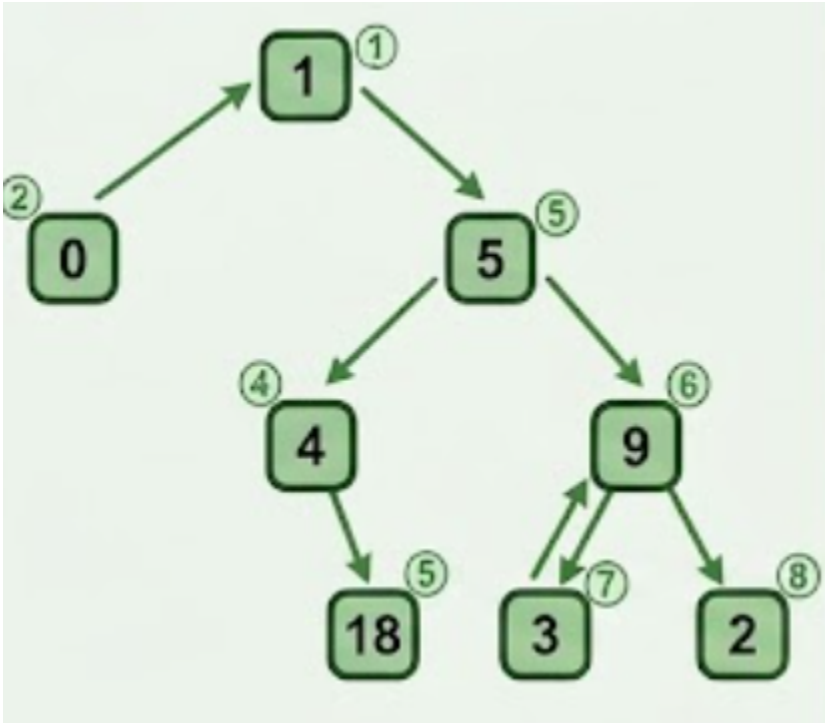
In the last **Task 3, Table 3.4**, The insertion function, where in we will be creating a new node that will contain the data value of "**O**" make this node the child of node G. The **findNode(this, parentKey)** is implemented since the function cannot predict where a node is located, thus, it passes "**this**" which is the root node to scan every branch until it finds the node containing **parentKey (G)**. As what I implemented in the earlier general tree structure, since I treat a certain child as a subroot, therefore, it is easier to access it as a targetKey, and process in the insertion below in. We just need to initialize a condition that if the parentNode is not equal to nullptr, it will run the add child method to add the new child into that parent's interval child-sibling linked list. Then return true to signal the caller that the insertion was successful.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

TREE TRAVERSAL DIAGRAMS
PRE-ORDER  **IN-ORDER**



POST-ORDER



Pre-order Traversal

[Function]

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if(left != nullptr){
        left->preOrder();
    }
    if (right != nullptr){
        right->preOrder();
    }
}
```

[Console Output]

```
preOrder: 2 0 1 3 9 4 5 18
```

[Analysis]

The pre-order traversal function follows the **ROOT->LEFT->RIGHT sequence** where the algorithm processes the **current (root/parent) node** first, and then recursively visits all nodes in the **left subtree**, followed by all nodes in the **right subtree**.

In-order Traversal

[Function]

```
template <typename T>
void BinarySearchTree<T>::inOrder(){
    // go to left
    if(left != nullptr){
        left->inOrder();
    }
    std::cout << key <<" ";
    //go to right
    if (right != nullptr){
        right->inOrder();
    }
}
```

[Console Output]

```
inOrder: 0 1 2 3 4 5 9 18
```

[Analysis]

The in-order Traversal follows the **LEFT->ROOT->RIGHT** where it recursively visits the left subtree, then the current node, and finally the right subtree. For any standard Binary Search Tree, an in-order traversal always processes values in **strictly ascending order**.

Post-order Traversal

[Function]

```cpp
template <typename T>
void BinarySearchTree<T>::postOrder(){
    // go to left
    if(left != nullptr){
        left->postOrder();
    }
    //go to right
    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key <<" ";
}
```

[Console Output]

```
postOrder: 1 0 5 4 18 9 3 2
```

[Analysis]

The post traversal function follows the **LEFT->RIGHT->ROOT** sequence, as it visits all nodes in the **left subtree**, processes the **current (root/parent) node**, and then recursively visits all nodes in the **right subtree**.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.